



Regular Expression :-

library

→

re



Use for detecting specific patterns and validate it.

Most Frequently used function .search() :-

re. search (pattern , string , flag = 0)

↙ string given to be check.

↘ pattern suggested by user

↘ Modifications in your code.

Some Symbols :-

• any character except a newline

* 0 or more repetitions

+ 1 or more repetitions.

? 0 or 1 repetition

{m} m repetition

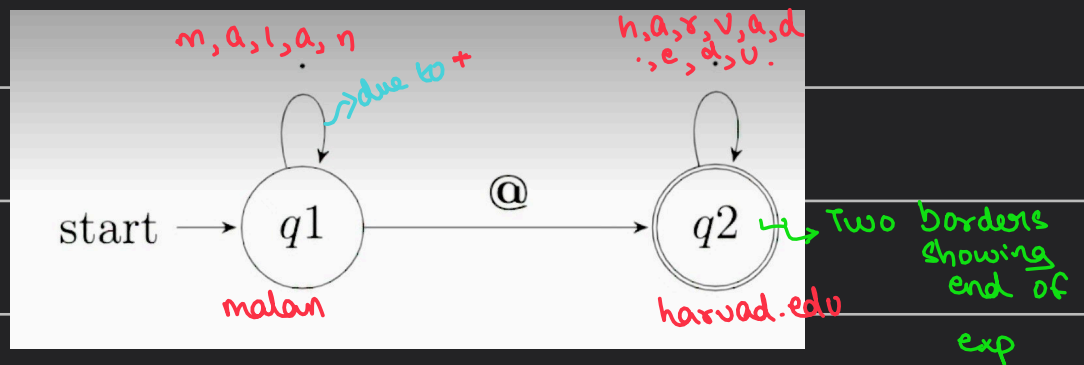
{m,n} m - n repetition

① Code :-

```
1 import re
2
3 email = input("What's your email? ").strip()
4
5 if re.search("[.*@.*", email):
6     print("Valid")
7 else:
8     print("Invalid")
```

Machine :-

• → showing each character.
* → should be repeated 0 or more.



eg:-

malan @ harvad . edu .

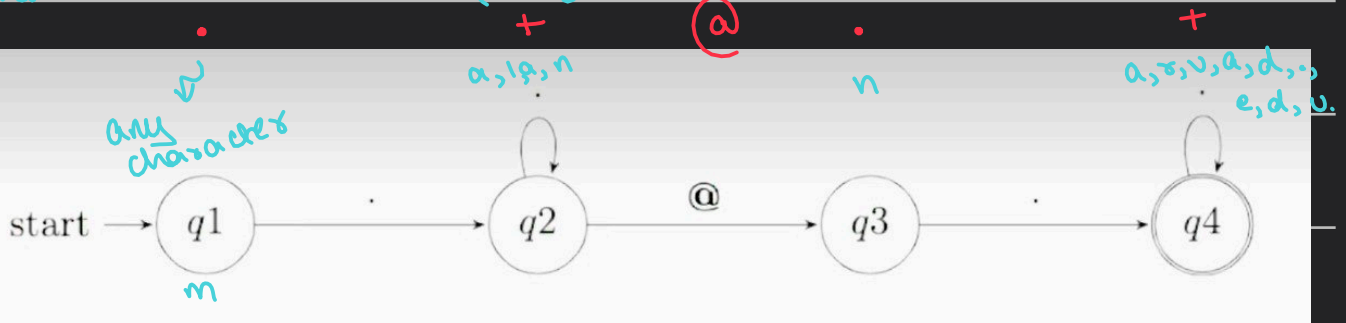
② Code :-

```
1 import re
2
3 email = input("What's your email? ").strip()
4
5 if re.search(".*+@.*+", email):
6     print("Valid")
7 else:
8     print("Invalid")
```

• → Any character
+ → 1 or more character repetition.

Machine :-

> 1 or more character → can be written as (.*)



To add a check for ".edu" we use \.edu with raw (r) string so not Pythonic brains run like \n $\leadsto$ means new line in python but I want n to be there in email \n and (r) before quotes.

`r".+ @ .+ \.edu"`

Means I literally want "." over here other than any other character.

$\wedge$ $\leadsto$ matches the start of string.

$\$$ $\leadsto$ matches the end of string or just before the newline at the end of the string.

$\leadsto$ `r".+ @ .+ \.edu $"`
 $\rightarrow$ any word "." over here.
 $\hookrightarrow$ want edu at the end.

[] $\leadsto$ specify set of characters

→ Means other than the special characters.

[^] → Complementing the set.

Interpretation :-

^① ^ ^② [^@] ^③ + ^④ @ ^⑤ [^@] ^⑥ + ^⑦ \. ^⑧ edu ^⑨ \$

① ^ → Start should contain the things on left.

② [^@] → Every character other than @

③ + → 1 time [^@] or more time.

④ @ → needed @.

⑤ [^@] → Every character other than @

⑥ + → 1 time [^@] or more time.

⑦ \. → Need "." over there.

⑧ edu → Needed edu over there.

⑨ \$ → End should have thing on

right.

a - z → all characters from a to z

Similar for A - Z , 0 - 9

^ [a-zA-Z0-9_] + @ [a-zA-Z0-9_] + \. edu \$

↓ can be written like this short.

\w

↓

$r"^\wedge \backslash w + @ \backslash w + \backslash . edu \$"$

\d → Decimal digit

\D → not a decimal digit

\s → whitespace characters.

\S → not a whitespace character.

\w → word character ... as well as numbers and the underscore.

\W → not a word character.

A|B → either A or B

(....) → a group.

$r"^\wedge \backslash w + @ \backslash w + \backslash . (edu|com|gov) \$"$

re. IGNORECASE

re. MULTILINE

re. DOTALL

re.search(r"^\w+@\w+\.\edu\$", email, re.IGNORECASE)

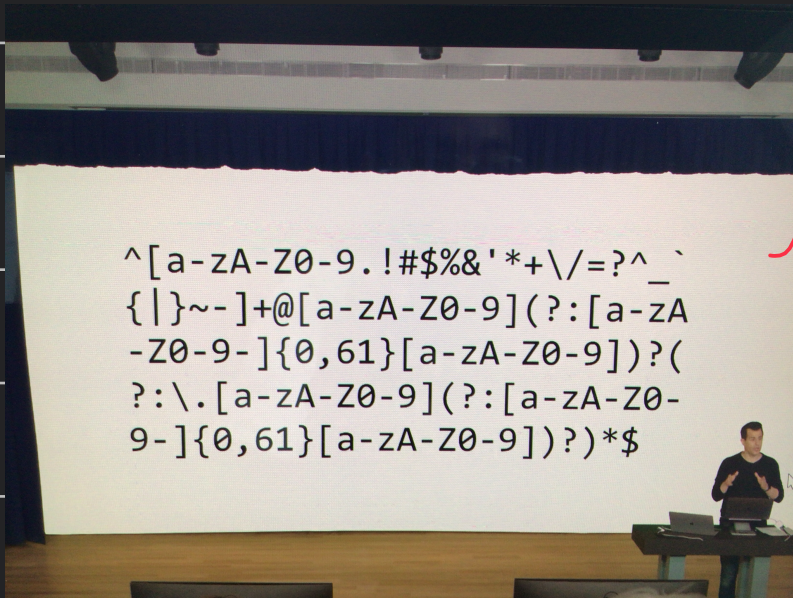
Having subdomain

$$r"^\w+@(\w+\.)?\w+\.\edu$"$$

either 0 or 1

dot in username:-

$$r"^\w|\w+\.)+@(\w+\.)?\w+\.\edu$"$$



official regex of email.

→ same as search.

re.match

↳ match from the start of the string. So no need to write ^ symbol.

→ same as search,

re.fullmatch

↳ match from the start and end of the string. So, need to write ^ & \$.

Code :-

```
1 import re
2
3 name = input("What's your name? ").strip()
4 matches = re.search(r"^(.+), (.+)$", name)
5 if matches:
6     ...last, first = matches.groups()
7     ...name = f"{first} {last}"
8 print(f"hello, {name}")
```

↪ return in return values
↪ return groups

re.search → return value
of
parenthesis.

if matches:

last = matches.group(1)

first = matches.group(2)

or

name = matches.group(2) + " " + matches.group(1)

indexing → starts at 1.

Want to assign and ask question simultaneously:-

if matches := re.search(r"(.+), *(.+)\$", name):

re.sub(pattern, repl, string, count = 0, flags = 0)

```
username = re.sub(r"https://twitter.com/", "", url)
```

```
re.sub(r"^(https?://)?(www\.)?twitter\.com/", "", url)
```

